1 Lesson 06-11-26

1.1 Processor peripherals

Let's connect the peripherals we have seen to a processor core. Different connections to the core are possible:

- Within the pipeline of the processor;
- Embedded on the same integrated circuit, with an On-Chip Bus;
- External circuit mapped in a shared memory;
- External circuit with communication protocol.

1.1.1 In-pipeline

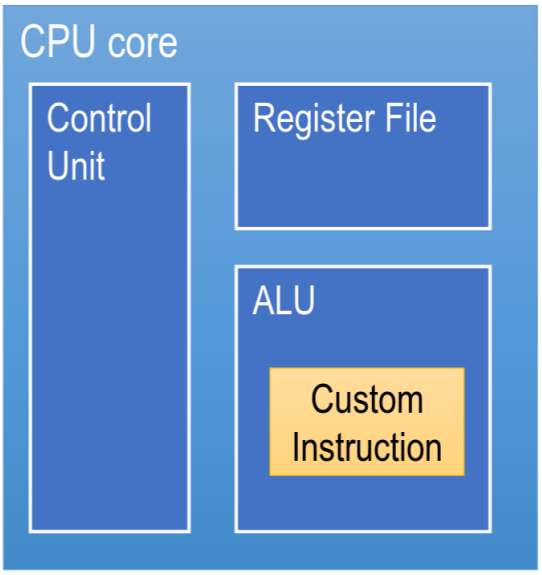
Peripherals *within* the processor pipeline are the absoulte fastest. They are defined as ustom instructions (not really a peripheral), so instructions dedicated to a specific application domain.

This gives a device direct access to the processor register file, and the lowest communication overhead. Note that this can only transfer a limited amount of data.

For specific needs, the processor must also be configurable (and for hard processors this means refabricating). *Soft processors*, that is processors fully described in a hardware description language, are easier for this soft Processors (*Nios II*).

Other kinds of processors can benefit for this, such as:

- Embedded FPGA (*XiRisc*);
- Soft Processor targeting hardware (*Tensilica Xtensa*).

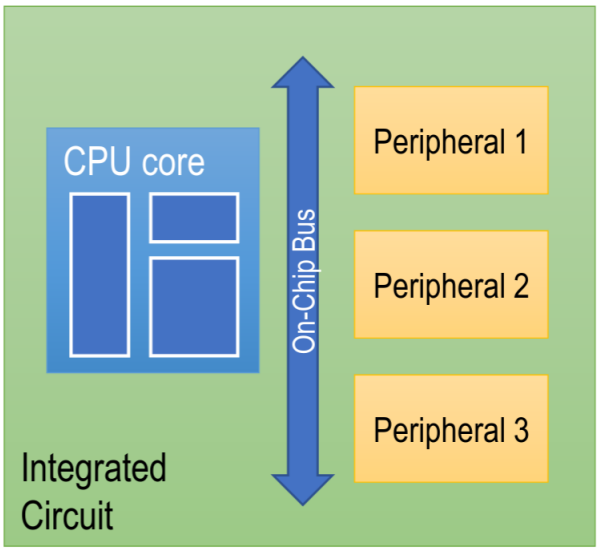


1.1.2 On-chip

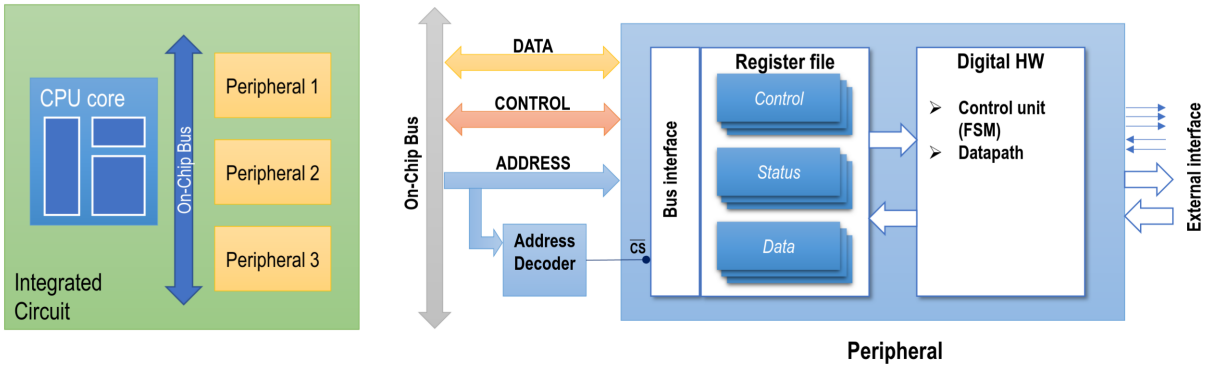
The next best solution (in terms of speed) is peripheral on the same chip, but outside the core. This means the peripheral is embedded in the same integrated circuit (**SoC**, *System on Chip*).

The peripheral is connected to an On-Chip Bus (such as *AMBA, Avalon, AXI, Network on Chip (NoC)*). The bus is accessed by the processor, either I/O Mapped or memory mapped.

There is low communication overhead, as data does not travel outside the integrated circuit (on pins). Of course a set of peripheral registers to exchange data with the processor core must be defined.



We want focus on this.



The typical architecture of single SoC peripheral is as shown above. The processor seems a number of peripherals on his on-chip-bus (which may be distinct from the local bus of the CPU core).

The peripheral takes from the bus the lines:

- **Data**,
- **Address**,
- **Control**.

Communication between processor core and peripheral only through the peripheral register file. The only exception is the address line, which is decoded and functions as an enable for the peripheral (at least its highest part, the low part decides among registers). The other lines are used differently, accessing the register file, based on read/write accesses:

- **Data**: gives data provided by the peripheral on read, takes data from processor on write;
- **Control**: gives status register on read, takes control bits on write.

Internally, the peripheral is designed in whatever way (usually FSM controlling a datapath).

The processor core is usually the master of the on-chip bus. A master, whatever the standard, can initiate a bus transaction to transfer data. The peripheral is a slave, it cannot generate transactions (we saw things similar to this for SPI and I2C).

So, the processor core generates bus transactions to access the peripheral register file. The address decoder enables accessing the peripheral when necessary, driving the chip select, and then selects registers within the file. Multiple bus transactions are required to access multiple registers.

Note that the On-Chip Bus is a shared resource: multiple blocks may drive the same wires concurrently, so there's risk of conflicts and possibly short circuits.

We need to prevent concurrent driving of wires.

This is be done through:

- **Arbitration** between *masters*: only a single master can access the bus at any given time instant;
- **Selection** of *slaves*: using addresses and possibly other control signals, feeding ddress decoders driving chip select.

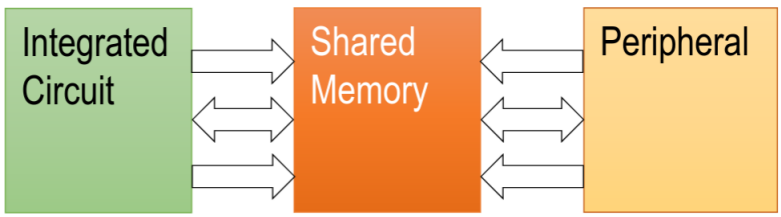
Different driving techniques are used electrically.

- For instance, all drivers can use tri-state output buffers: a single one is enabled, the others are in tri-state.
- Another solution is that a multiplexer drives the shared resource, a single driver output is selected. This way there's no need for tri-state outputs.

1.1.3 External-chip, shared memory

This is the case of a peripheral on an external chip with shared memory. We need an external integrated circuit or FPGA. It's the case of oprocessors (e.g., *JPEG or MPEG codec*), application specific digital circuits designed to work alongisde a procssor.

Shared memory can be used for communication, usually large, on the same chip of the peripheral, and multiport. The processor core often needs a memory controller (embedded peripheral) to access the shared memory. There his high communication overhead, usually SRAM is used, and peripheral usually implements a costly and complex function to make the communication overhead acceptable.



1.1.4 External-chip, no shared memory

The same thing can be done without shared a memory. This time, instead of a memory access protocol to touch the peripheral's memory, we need a standard communication

protocol such as *SPI, I2C, I2S, UART, USB, PCI, PCIExpress, SATA*. Sometimes even a custom communication protocol can be used.

There is high communication overhead, a custom parallel protocol is sometimes used to increase communication performance.

